

CIS270A Project 10

In this project, you will intentionally create some error conditions that would normally result in an abnormal end or runtime error, and practice techniques of handling these errors.

Part One: Proj10A – using (e) extender and %Error

1. Copy your CIS270A Project #2 into a new source member called PROJ10A
2. Re-read the project specs for this project (in the Update Database unit) to re-familiarize yourself with its purpose
3. Set up subroutines to contain the “good chain” logic and the “bad chain” logic and move the code into these subroutines. Update your IF to use these subroutines.
4. Refresh your copy of the master file:

```
DLTF YOURLIB/MYCSTP  
CPYF FROMFILE(RPGIV_5THS /GTCSTP) TOFILE(yourllibrary/MYCSTP) CRTFILE(*YES)
```
5. Call your program and it should behave exactly as it did before.
6. Next, we will intentionally cause an error in the program that will be caught by using the (e) extender and the %Error built-in-function.
7. First, add another error line to your printer file that will print ‘Input/Output Error – Program will Terminate’
8. In your RPG program, right before your chain statement, insert `CLOSE MYCSTP;`
9. Modify your chain operation to include the (e) extender
10. Code a subroutine that prints the “Input/Output” error line and then terminates the program (simply by setting LR on and doing a Return)
11. Use %Error to execute this subroutine.
12. Call your program; it should print the error message from your new error subroutine.

Part Two: Proj10B – using Monitor with generics (*File and *All)

1. Copy your Proj10A into Proj10B
2. Add the code to encompass your mainline (the code before your subroutines) in a Monitor block. Your Endmon should be just before the existing *INLR=ON; and Return; statements.
3. Add On-error code – one that monitor for *File errors and executes the same subroutine that you made in Part One, and another that monitors for *All that executes another subroutine that you will create that will print an error of your choice.
4. To prove that your new Monitor block is catching the problem of the closed file, and not your (e) extender and %Error, remove the (e) extender from your chain and comment out your If %Error code.
5. Call your program; it should produce a report with your “I/O Error” message on it.

Part Three: Proj10ERR – specific status codes with Monitor

1. Create a simple report that contains the following:
 - a. A heading “Playing with Monitor”
 - b. An error line that will print “Divide by Zero Not Allowed”
 - c. An error line that will print “Array Index is Out of Bounds”
 - d. An error line that will print “Variable is too small”
2. Next, open up PROJ10ERR
3. Look up the status codes for the error conditions.
4. Create named constants for your status codes as shown on Page 500.
5. Place each block of offending code in a Monitor block with the appropriate On-error statements that cause your error line to print.

Do your housekeeping and email when done.